

Experiment 1: Preprocessing Impact Analysis

Aim

To study the impact of noise removal and normalization on image quality and feature detection.

Requirements

- Python
- OpenCV
- NumPy
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Add noise to the image.
4. Remove noise using Gaussian Blur.
5. Normalize the image.
6. Apply Canny edge detection before and after preprocessing.
7. Compare the results.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Add noise

noise = np.random.normal(0, 25, gray.shape)

noisy = np.clip(gray + noise, 0, 255).astype(np.uint8)
```

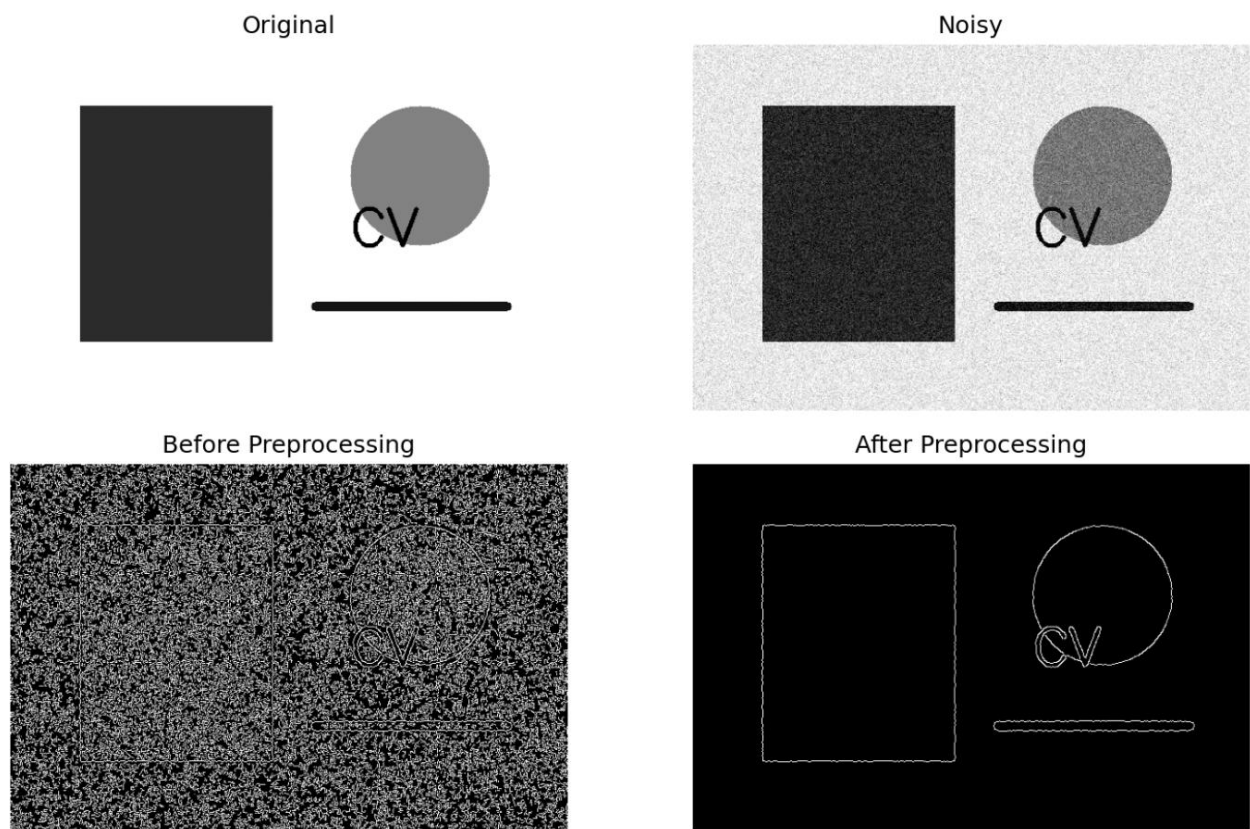
```
# Noise removal
filtered = cv2.GaussianBlur(noisy, (5, 5), 0)

# Normalization
normalized = cv2.normalize(filtered, None, 0, 255, cv2.NORM_MINMAX)

# Edge detection
edges_before = cv2.Canny(noisy, 100, 200)
edges_after = cv2.Canny(normalized, 100, 200)

plt.figure(figsize=(10, 6))
plt.subplot(2, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(noisy, cmap="gray")
plt.title("Noisy")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(edges_before, cmap="gray")
plt.title("Before Preprocessing")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(edges_after, cmap="gray")
plt.title("After Preprocessing")
plt.axis("off")
plt.show()
```

Output



Result

The preprocessing techniques successfully reduced noise and produced cleaner and more reliable edge detection results.

Experiment 2: Image Representation Study

Aim

To convert an image into **grayscale and binary representations** and study their effect on edge detection and feature extraction.

Requirements

- Python
- OpenCV
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert the image into grayscale.
3. Convert the grayscale image into binary using thresholding.
4. Apply edge detection to both representations.
5. Display and compare the results.

Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert to binary
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

```
# Edge detection
gray_edges = cv2.Canny(gray, 100, 200)
binary_edges = cv2.Canny(binary, 100, 200)

# Display
plt.figure(figsize=(10, 6))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

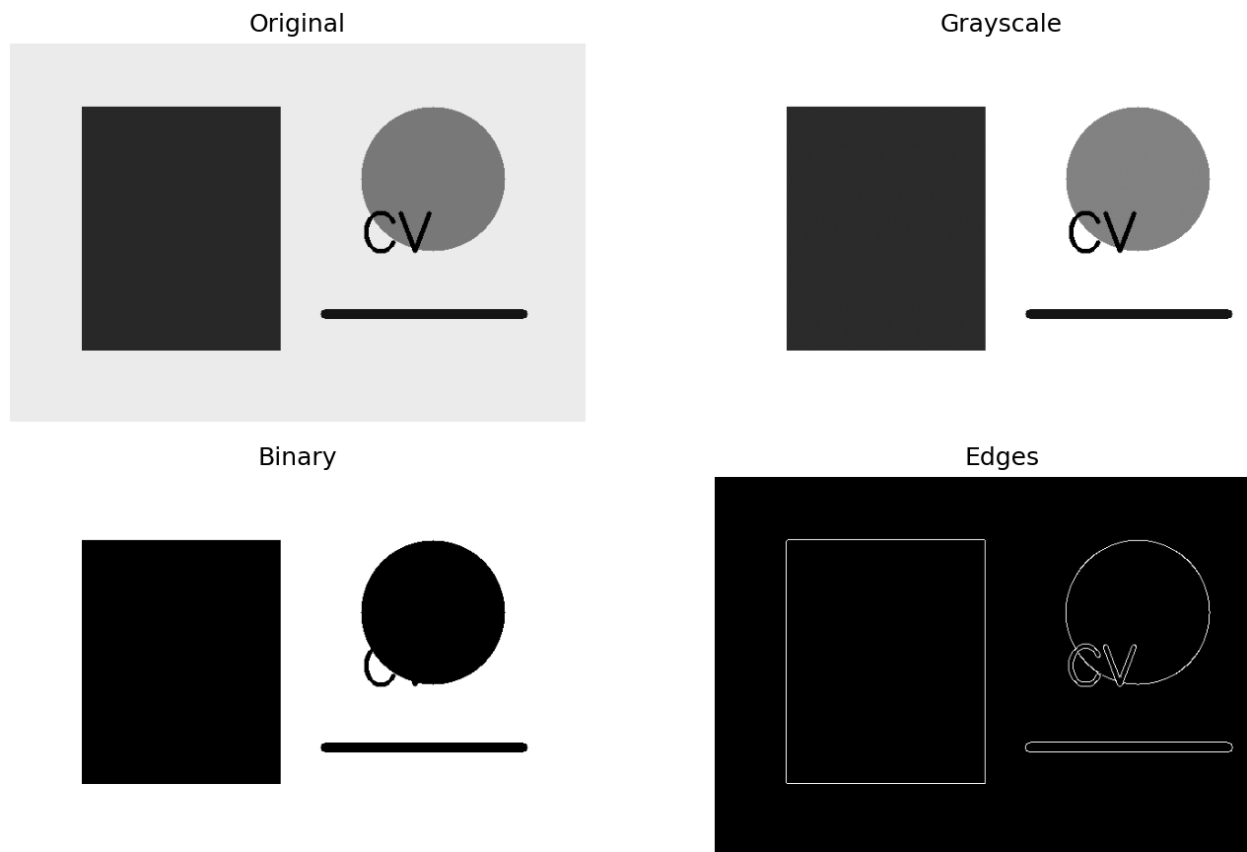
plt.subplot(2, 2, 3)
plt.imshow(binary, cmap="gray")
plt.title("Binary")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(gray_edges, cmap="gray")
plt.title("Edge Detection")
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output



Result

The image was successfully converted into **grayscale and binary representations**. Grayscale preserves intensity information, while binary represents the image using two intensity levels, making it useful for segmentation and shape analysis

Experiment 3: Edge Detection Comparison

Aim

To compare **Sobel and Canny edge detection** techniques on the same image.

Requirements

- Python
- OpenCV
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Sobel edge detection.
4. Apply Canny edge detection.
5. Display and compare the results.

Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel edge detection
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)

# Canny edge detection
```

```
canny = cv2.Canny(gray, 100, 200)
```

```
# Display
```

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(sobel, cmap="gray")
```

```
plt.title("Sobel Edge Detection")
```

```
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(canny, cmap="gray")
```

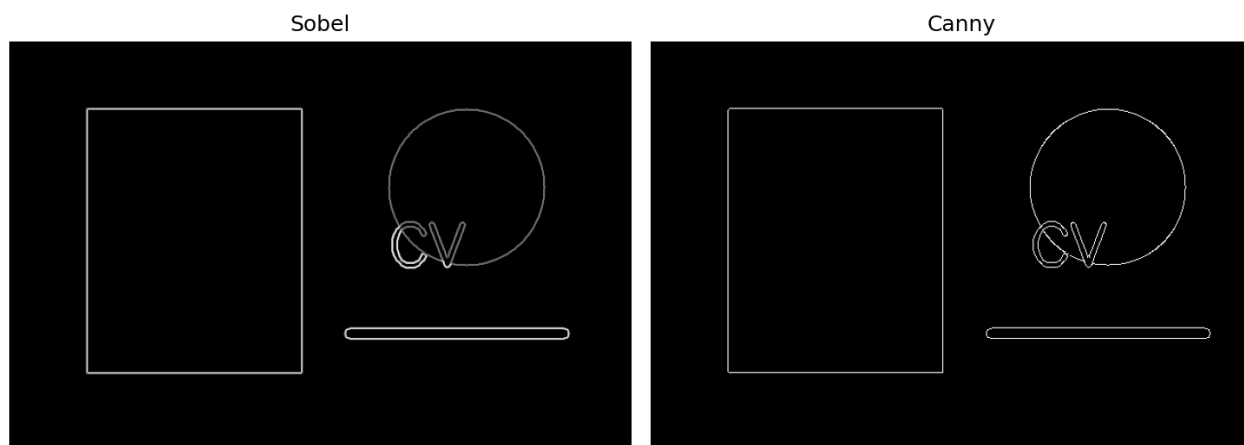
```
plt.title("Canny Edge Detection")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output



Result

Sobel and Canny edge detection were successfully applied. **Canny generally produces cleaner and more precise edges**, while Sobel is simpler and useful for detecting intensity gradients.

Experiment 4: Harris Corner Detection

Aim

To perform **Harris corner detection** on an image and analyze the detected corner points.

Requirements

- Python
- OpenCV
- NumPy
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Harris Corner Detection.
4. Mark the detected corner points.
5. Display the output image.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Harris corner detection

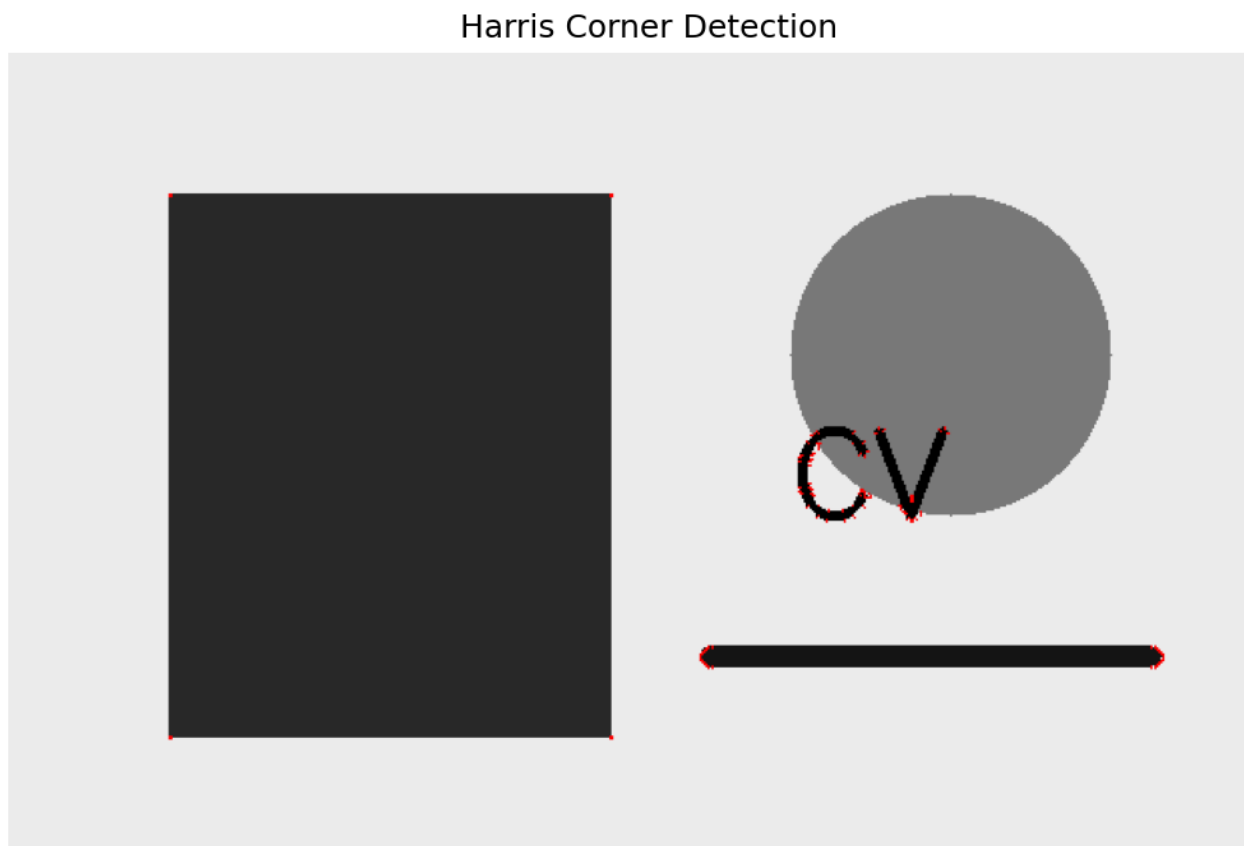
gray = np.float32(gray)

corners = cv2.cornerHarris(gray, 2, 3, 0.04)
```

```
# Mark corners
img[corners > 0.01 * corners.max()] = [0, 0, 255]

# Display
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Harris Corner Detection")
plt.axis("off")
plt.show()
```

Output



Result

Harris Corner Detection was successfully performed, and the **corner points of the image were detected and highlighted.**

Experiment 5: Hough Transform for Shape Detection

Aim

To detect **lines or circles** in an image using the Hough Transform.

Requirements

- Python
- OpenCV
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply edge detection.
4. Apply Hough Line Transform.
5. Detect and draw the lines.
6. Display the output.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Edge detection

edges = cv2.Canny(gray, 50, 150)

# Hough Line Transform
```

```
lines = cv2.HoughLinesP(  
    edges, 1, np.pi / 180,  
    threshold=100,  
    minLineLength=50,  
    maxLineGap=10  
)
```

```
# Draw detected lines
```

```
if lines is not None:
```

```
    for line in lines:
```

```
        x1, y1, x2, y2 = line[0]
```

```
        cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 2)
```

```
# Display
```

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

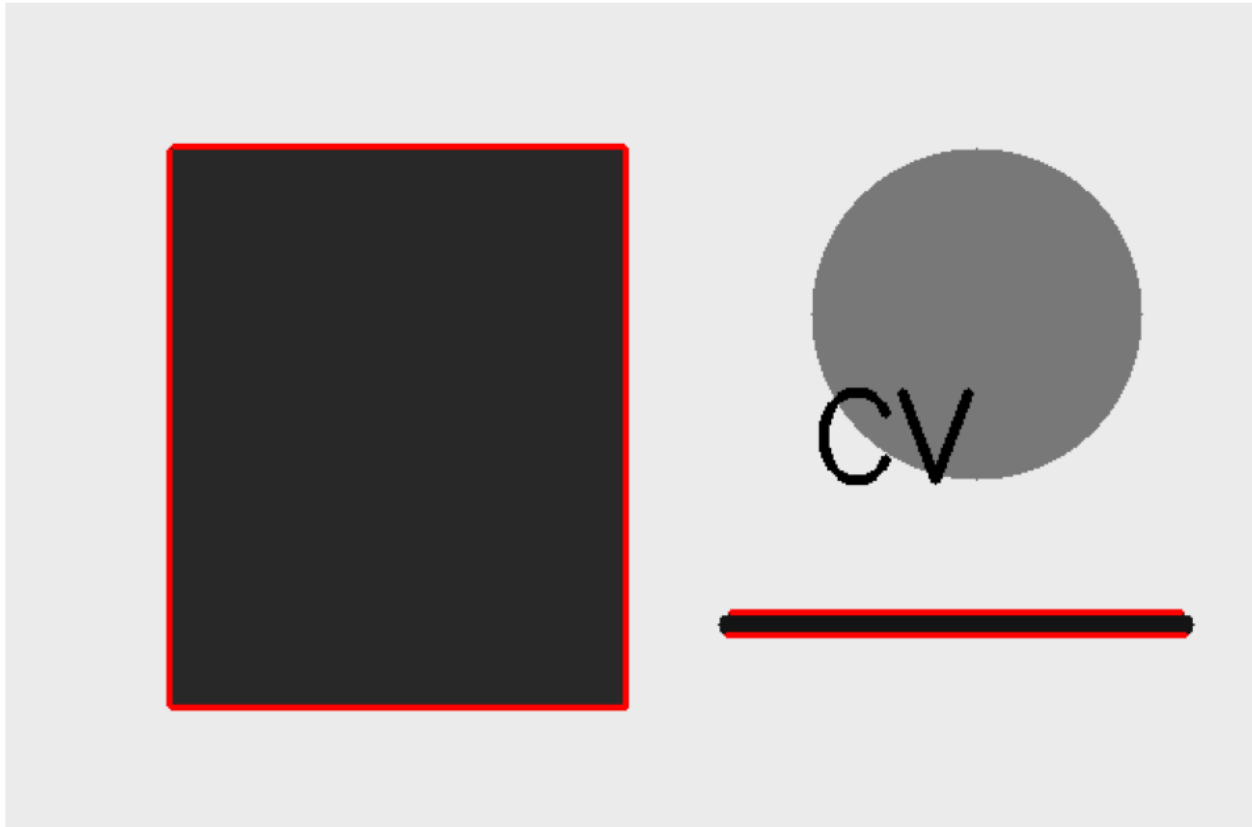
```
plt.title("Hough Line Detection")
```

```
plt.axis("off")
```

```
plt.show()
```

Output

Hough Line Detection



Result

The Hough Transform was successfully used to **detect and highlight lines** present in the input image.

Experiment 6: Feature Matching using SIFT

Aim

To perform feature extraction and matching between two images using the **SIFT algorithm**.

Requirements

- Python
- OpenCV
- Matplotlib
- Two input images

Procedure

1. Read two images.
2. Convert both images to grayscale.
3. Create a SIFT detector.
4. Detect keypoints and descriptors.
5. Match the descriptors using a Brute-Force matcher.
6. Display the matched features.

Code

```
import cv2

import matplotlib.pyplot as plt

img1 = cv2.imread("image1.jpg")
img2 = cv2.imread("image2.jpg")

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()
```

```

# Detect keypoints and descriptors
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

# Brute-Force matcher
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)

# Apply ratio test
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

# Draw matches
result = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    good_matches, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

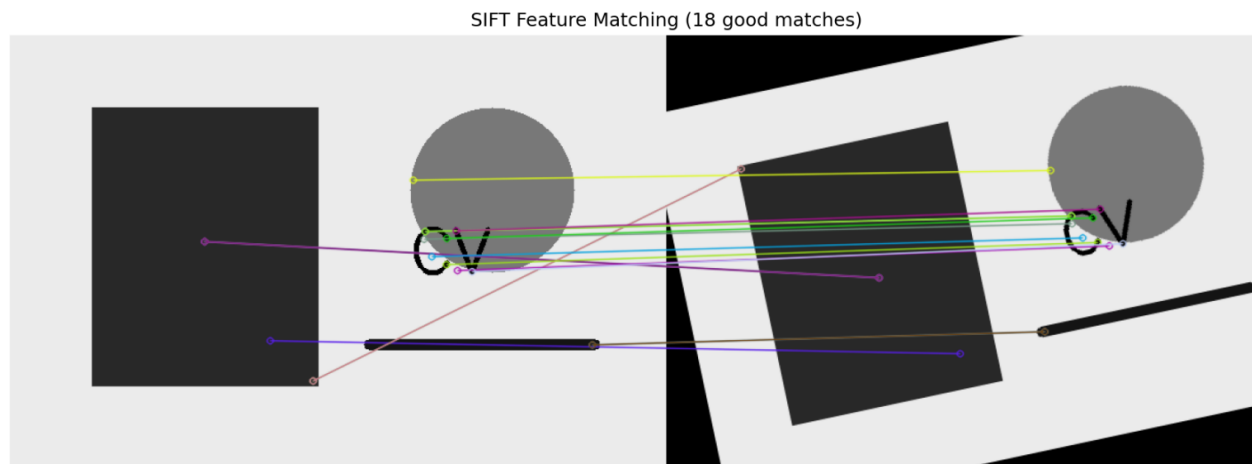
plt.figure(figsize=(12, 6))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Matching")

```

```
plt.axis("off")
```

```
plt.show()
```

Output



Result

SIFT feature extraction and matching were successfully performed between two images. The **matching keypoints** were detected and displayed.

Experiment 7: Effect of Smoothing on Edge Detection

Aim

To study the effect of **smoothing techniques on edge detection** by reducing noise and improving edge continuity.

Requirements

- Python
- OpenCV
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Gaussian smoothing.
4. Apply Canny edge detection before and after smoothing.
5. Compare the results.

Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Edge detection before smoothing
edges_before = cv2.Canny(gray, 100, 200)

# Apply Gaussian smoothing
smooth = cv2.GaussianBlur(gray, (5, 5), 0)
```

```

# Edge detection after smoothing
edges_after = cv2.Canny(smooth, 100, 200)

# Display results
plt.figure(figsize=(10, 5))

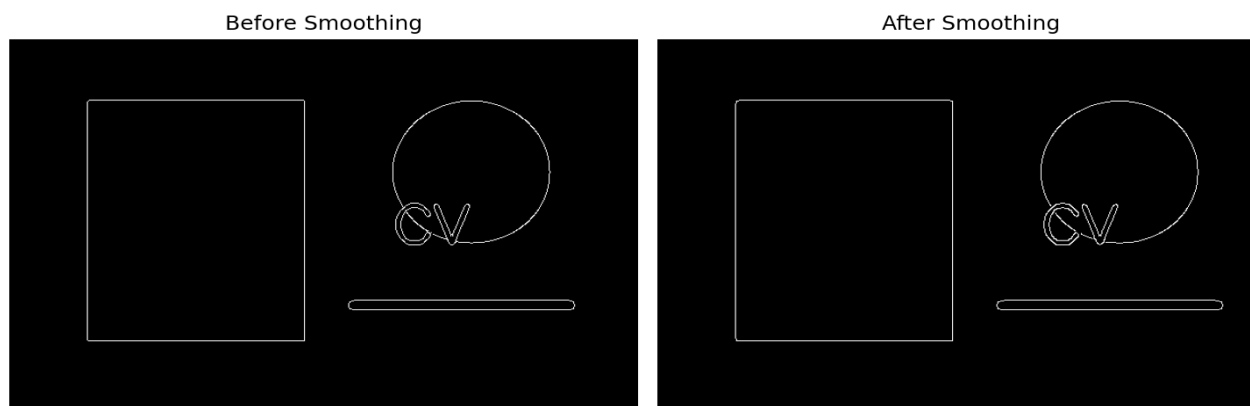
plt.subplot(1, 2, 1)
plt.imshow(edges_before, cmap="gray")
plt.title("Before Smoothing")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(edges_after, cmap="gray")
plt.title("After Smoothing")
plt.axis("off")

plt.tight_layout()
plt.show()

```

Output



Result

Smoothing successfully reduced image noise and produced **cleaner and more continuous edges** during edge detection.

Experiment 8: Prewitt vs Sobel Comparison

Aim

To compare **Prewitt and Sobel edge detection operators** on the same image.

Requirements

- Python
- OpenCV
- NumPy
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply the Prewitt operator.
4. Apply the Sobel operator.
5. Display and compare the detected edges.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Prewitt kernels
prewitt_x = np.array([[ -1, 0, 1],
                      [ -1, 0, 1],
                      [ -1, 0, 1]])
```

```
prewitt_y = np.array([[ -1, -1, -1],  
                      [ 0, 0, 0],  
                      [ 1, 1, 1]])
```

```
px = cv2.filter2D(gray, cv2.CV_64F, prewitt_x)  
py = cv2.filter2D(gray, cv2.CV_64F, prewitt_y)
```

```
prewitt = cv2.magnitude(px, py)
```

```
# Sobel
```

```
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)  
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

```
sobel = cv2.magnitude(sx, sy)
```

```
# Display
```

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(prewitt, cmap="gray")
```

```
plt.title("Prewitt")
```

```
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(sobel, cmap="gray")
```

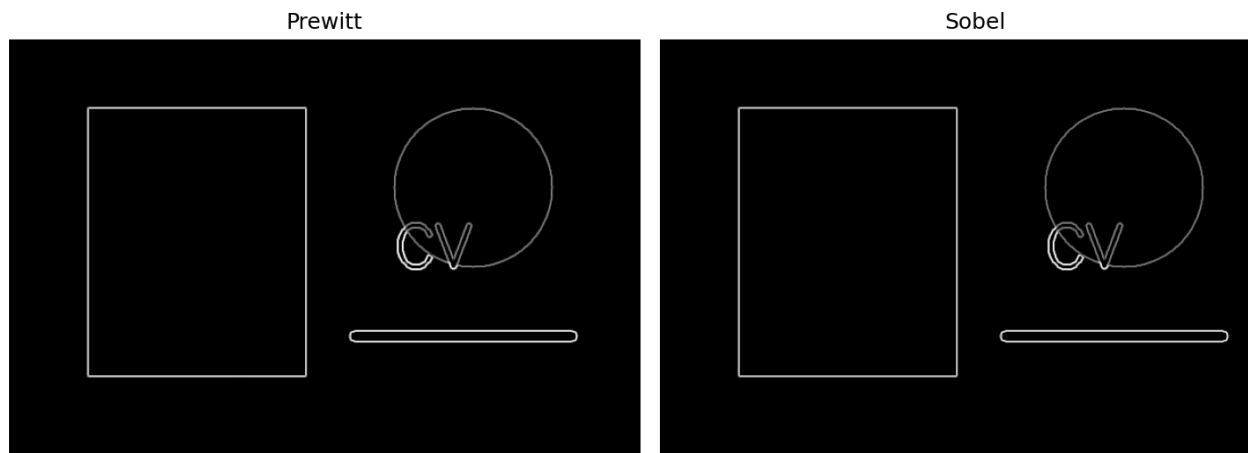
```
plt.title("Sobel")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output



Result

Prewitt and Sobel edge detection were successfully performed. Both methods detected image boundaries, with **Sobel generally providing stronger and smoother edge detection.**

Experiment 9: Threshold Effect in Edge Detection

Aim

To study the effect of different **threshold values on edge detection quality**.

Requirements

- Python
- OpenCV
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Canny edge detection with different threshold values.
4. Display and compare the results.
5. Analyze the effect of threshold values on edge detection.

Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges1 = cv2.Canny(gray, 50, 100)
edges2 = cv2.Canny(gray, 100, 200)
edges3 = cv2.Canny(gray, 150, 300)

plt.figure(figsize=(12, 4))
```

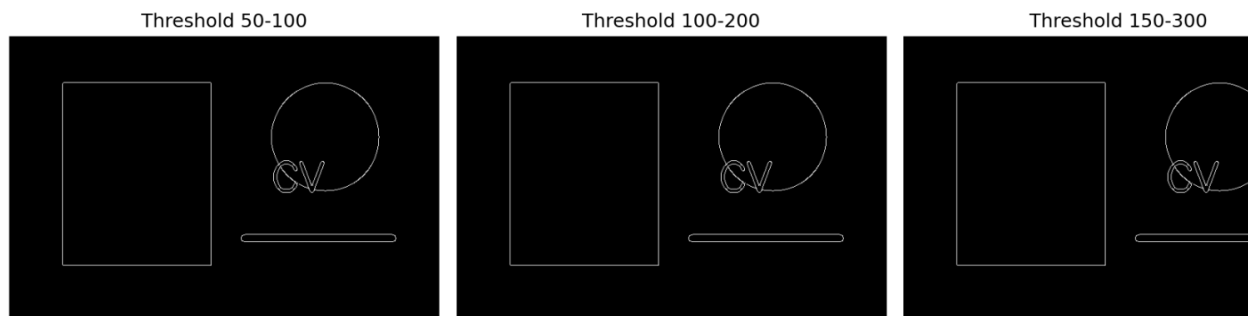
```
plt.subplot(1, 3, 1)
plt.imshow(edges1, cmap="gray")
plt.title("Threshold: 50-100")
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(edges2, cmap="gray")
plt.title("Threshold: 100-200")
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
plt.imshow(edges3, cmap="gray")
plt.title("Threshold: 150-300")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Output



Result

Different threshold values produced different edge detection results. **Lower thresholds detect more edges, while higher thresholds reduce weak and unwanted edges.**

Experiment 10: Noise Removal Comparison

Aim

To compare different **noise removal techniques** and analyze their effectiveness in preserving image details.

Requirements

- Python
- OpenCV
- Matplotlib
- Input image

Procedure

1. Read the input image.
2. Add noise to the image.
3. Apply Gaussian, Median, and Bilateral filters.
4. Display the filtered images.
5. Compare the noise reduction and image details.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Add noise

noise = np.random.normal(0, 25, gray.shape)

noisy = np.clip(gray + noise, 0, 255).astype(np.uint8)
```

```
# Noise removal  
gaussian = cv2.GaussianBlur(noisy, (5, 5), 0)  
median = cv2.medianBlur(noisy, 5)  
bilateral = cv2.bilateralFilter(noisy, 9, 75, 75)
```

```
# Display  
plt.figure(figsize=(12, 6))
```

```
plt.subplot(2, 2, 1)  
plt.imshow(noisy, cmap="gray")  
plt.title("Noisy Image")  
plt.axis("off")
```

```
plt.subplot(2, 2, 2)  
plt.imshow(gaussian, cmap="gray")  
plt.title("Gaussian Filter")  
plt.axis("off")
```

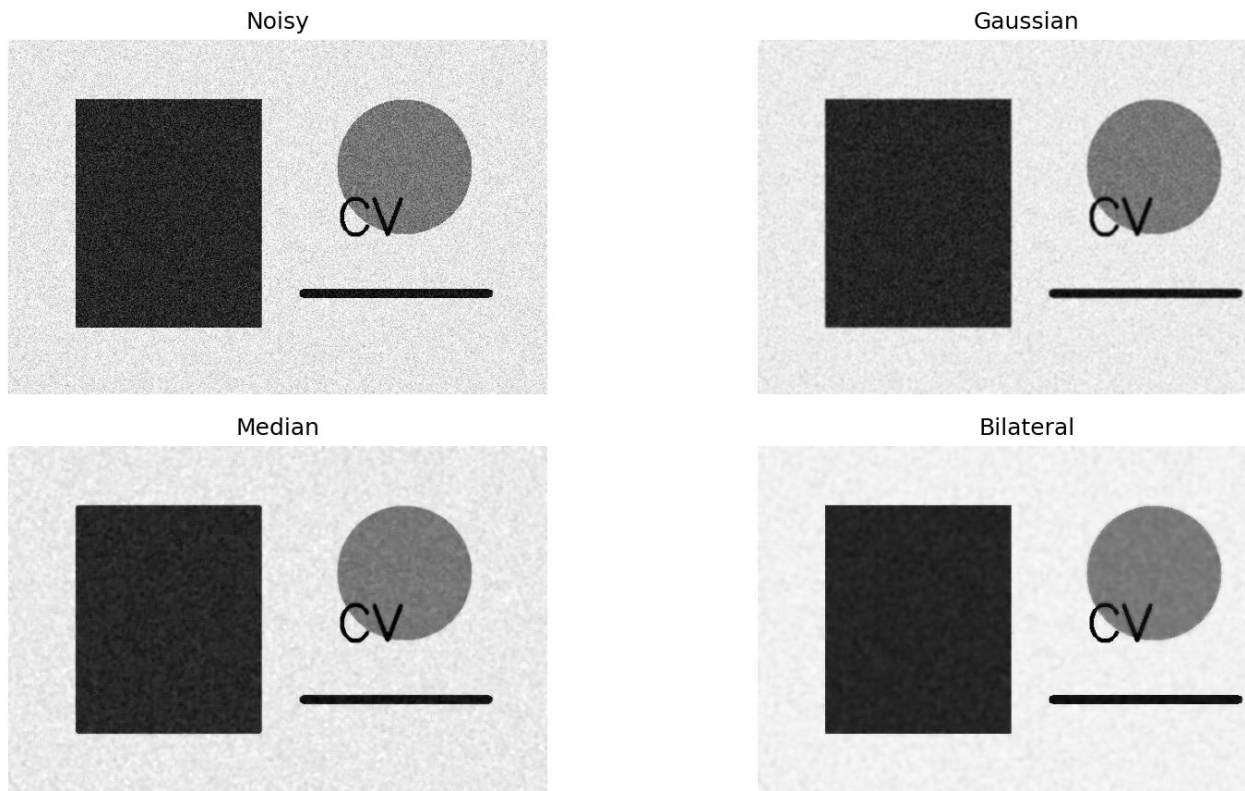
```
plt.subplot(2, 2, 3)  
plt.imshow(median, cmap="gray")  
plt.title("Median Filter")  
plt.axis("off")
```

```
plt.subplot(2, 2, 4)  
plt.imshow(bilateral, cmap="gray")  
plt.title("Bilateral Filter")  
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output



Result

Different noise removal filters were successfully applied. The filters reduced image noise while attempting to preserve important image details.